

g1_brain v1.1.0 — 完整运行时手册

本文件是 **v1.1.0 (May 2026)** 时刻的 g1_brain 端到端运行时快照。

与早期 `architecture.md` / `structure.md` 的区别：

- 那两份是 **设计意图** 文档；本份是 **代码实际行为** 的实证审计。
- 重点回答三个常被误解的问题：
 - 本地 **perception** 到底有没有进入快脑？
 - Codex** 到底在什么时刻被调用？
 - `recall_*` 是 LLM-in-the-loop 的工具代理吗？还是纯 Python？

所有结论都附 `file:line` 引用，可直接 `Ctrl+G` 验证。

目录

1.	一句话总览
2.	系统全景图
3.	进程 / 线程 / asyncio Task 全树
4.	启动序列 (agent main 的 7 个阶段)
5.	SceneStateBus + RobotStateBus: 核心数据契约
6.	快脑 (Fast Brain) : BrainRealtimeAgent
7.	慢脑 (Slow Brain) : Codex 的三种调用模式
8.	记忆系统: Phase 1 / Phase 2 / Recall
9.	工具表 (18 个 tool 的完整矩阵)
10.	安全监督: 11+1 规则 + FSM + Watchdog
11.	音频流与对话状态机
12.	一次完整 turn 的端到端时序
13.	关键问题回答
14.	已知局限与下一步
15.	电话桥 (Twilio + Realtime, 2026-05-24 增量)

1. 一句话总览

g1_brain v1.1.0 是一个三脑系统：

脑	频率	实现	何时工作
快脑	0.2-2 Hz / turn	OpenAI Realtime API (WebSocket, gpt-realtime)	全程在线，监听 mic、对话、决策、调用 tool

脑	频率	实现	何时工作
慢脑 (在线)	按需	codex mcp-server 子进程 (reasoning_effort=high, service_tier=fast)	仅当快脑显式调用 ask_slow_brain(query) 工具时触发
慢脑 (离线)	后台批处理	codex exec --json 一次性子进程 (Phase 1 + Phase 2)	session 结束后异步消化 jsonl → MEMORY.md

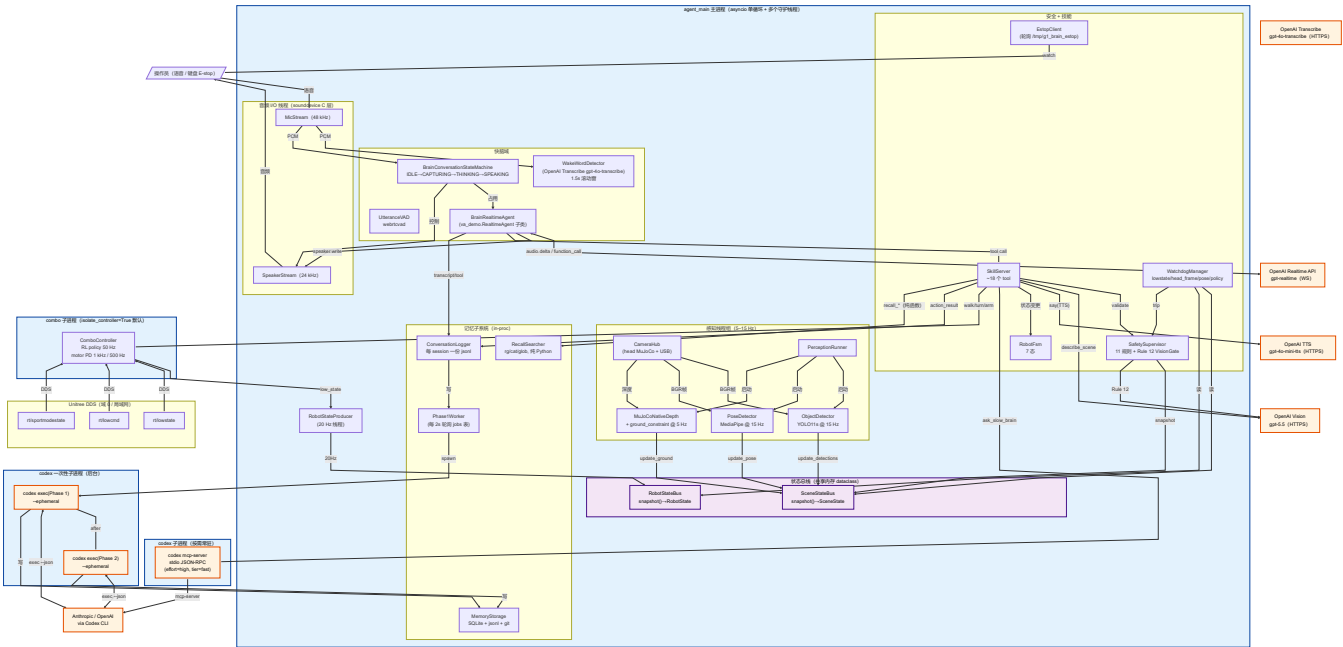
反射层 (Fast Reflex) 是另一回事：50 Hz RL policy + 20 Hz watchdog + 5 Hz perception，全部纯 Python，不含 LLM。

NEW 2026-05-24 增量：v1.1.0 之上加了 **电话桥** (gl_brain/phone/) ——一条**并列**的快脑入口，把 Twilio Voice Media Streams 桥接到一个独立的 OpenAI Realtime session，再走**完全相同**的 **safetySupervisor** + **vision_risk_gate** + **SkillServer** 实例。详见 §15。本节其它部分（三脑、反射层）一字未改。

- ⚠ 用户的两个直觉判断都正确：
- 快脑的本地 **perception** 几乎没有“持续流”进入 LLM —— 只在 motion tool 返回值里被注入一帧 **scene_after**，或被快脑主动调用 **query_scene_state()** / **describe_scene()** 时按需查询。**inject_perception_event()** 这条路径在 v1.1.0 **只对 mock_imitation 模块生效**，普通用户路径不走。
 - recall_grep / recall_read / recall_glob** 是纯 **Python**，由快脑在 realtime 里调用，但**不进 codex、不进任何 LLM**。它们只是带 sandbox 的 **rg/cat/glob**。“harness 的 memory recall 由 tool agent 实时执行”这个观察对的——但那个 tool 不是 LLM agent，是纯函数。

2. 系统全景图

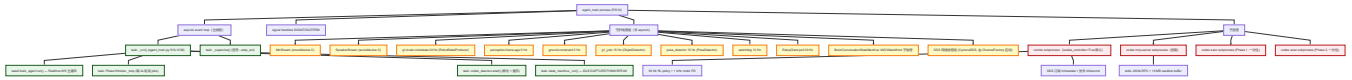
这张图把**所有**进程边界、所有 LLM、所有跨进程 IPC 都画出来。



这张图揭示的事实：

1. **唯一持续连接 LLM 的通路是** `RTA <-> OAI_RT`。所有其他 LLM 调用都是一次性的（vision / tts / codex exec）或按需的（codex mcp-server）。
2. **Perception 写入 SceneStateBus**，但从 `SceneBus → RTA` 之间**只有 SkSv 一条间接路径**——即"motion tool 返回 `scene_after`"或"快脑显式调用 `query_scene_state`"。
3. **快脑从不直接读 SceneStateBus。**

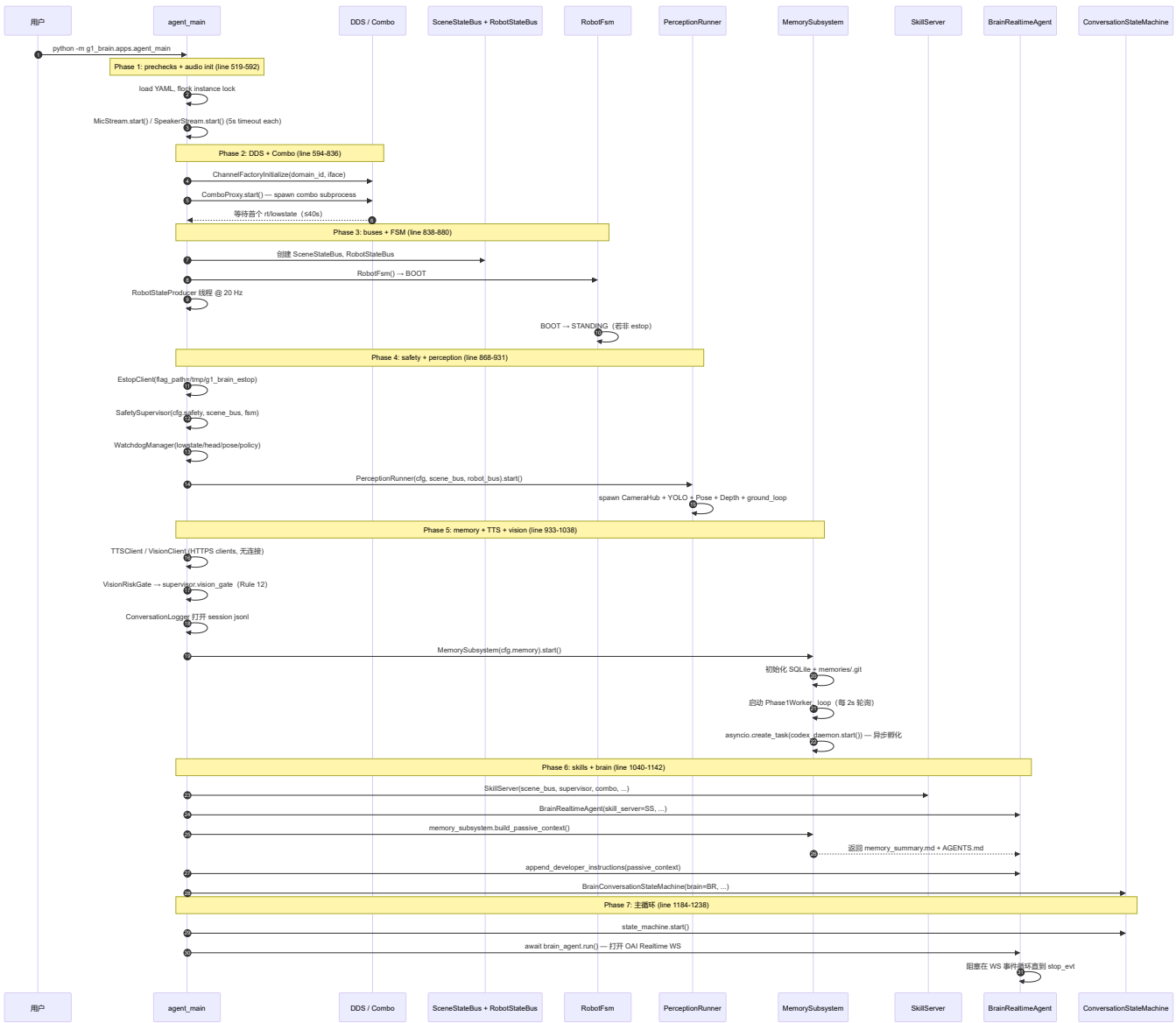
3. 进程 / 线程 / asyncio Task 全树



关键事实：

- 整个主程序是**单 asyncio event loop**，所有 await 都在主线程
 - DDS / 音频 / YOLO / pose 是**普通线程**，通过 GIL+共享 dataclass 与 event loop 通信
 - 50 Hz 控制循环**默认隔离到子进程**（`combo_proxy.py`），避免 perception 抢 GIL 拖垮 policy
 - Codex 的 **mcp-server 子进程是常驻**（除非 `memory.enabled=false`），exec-mode 子进程是**一次性**
-

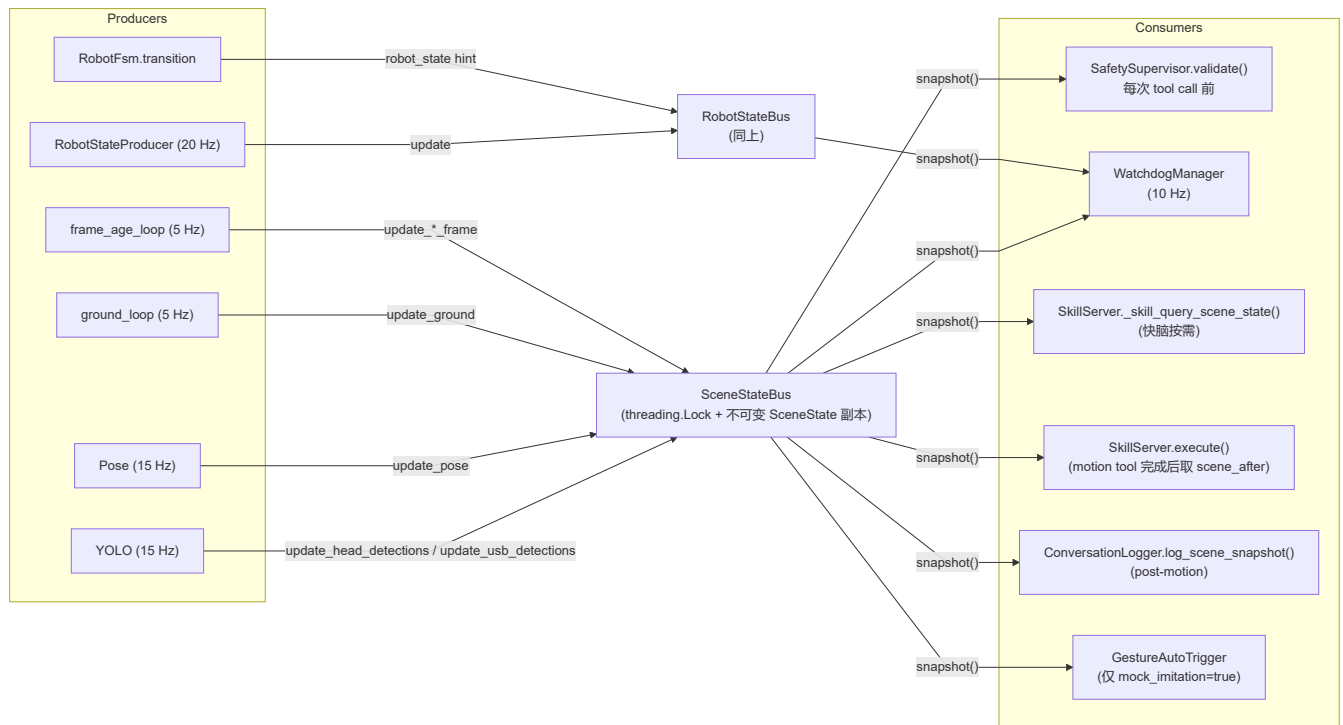
4. 启动序列（agent_main 的 7 个阶段）



关键不变量：**DDS 必须先于 CameraHub 初始化**（agent_main.py:595-598），因为头部相机内部会订阅 `rt/sportmodestate` 来同步根坐标到 MuJoCo 的合成头部相机。

5. SceneStateBus + RobotStateBus：核心数据契约

这是整个系统的"真相之源"。所有感知/状态写到这里，所有消费者（safety / watchdog / brain）从这里读。



核心观察：

- `snapshot()` 返回的是不可变 `dataclass` 副本（`scene_state/fusion.py`）；这是为了让 `safety` 不会读到半更新状态
- "快脑读 `SceneStateBus`" 这条边并不存在——快脑只能通过 `SkSv` 的 `query_scene_state` tool 间接拿到 `summary_for_llm()` 的精简 dict
- `summary_for_llm()` 返回的字段：`persons_visible`, `nearest_obstacle_m`, `nearest_person_m`, `clear_path`, `surface_tilt_deg`, `user_gesture`, `warnings`
- YOLO 原始检测、深度图、姿态骨架——快脑都看不到

6. 快脑（Fast Brain）：BrainRealtimeAgent

6.1 类继承与生命周期

这就是用户怀疑的核心点。结论：

通道	触发	频率	谁决定?	数据形态
A — <code>scene_after</code> 注入	motion tool 成功	每次 walk/turn/gesture/look_at 之后	自动	<code>summary_for_llm()</code> 精简 dict
B1 — <code>query_scene_state</code>	快脑判断需要	不确定 (取决于 LLM 决策)	LLM	同上
B2 — <code>describe_scene</code>	快脑判断需要	同上, 开销大 (vision API ~1-3s)	LLM	GPT-Vision 文字描述
C — <code>inject_perception_event</code>	mock_imitation 触发	仅在 <code>mock_imitation.enabled=true</code>	GestureAutoTrigger	system message 文本

v1.1.0 没有"持续推送 perception 给快脑"的通路。快脑对世界的"持续感知"完全是通过它自己决定调用 `query_scene_state` 的频率——也就是说，LLM 只在它主动想看的时候看一眼。

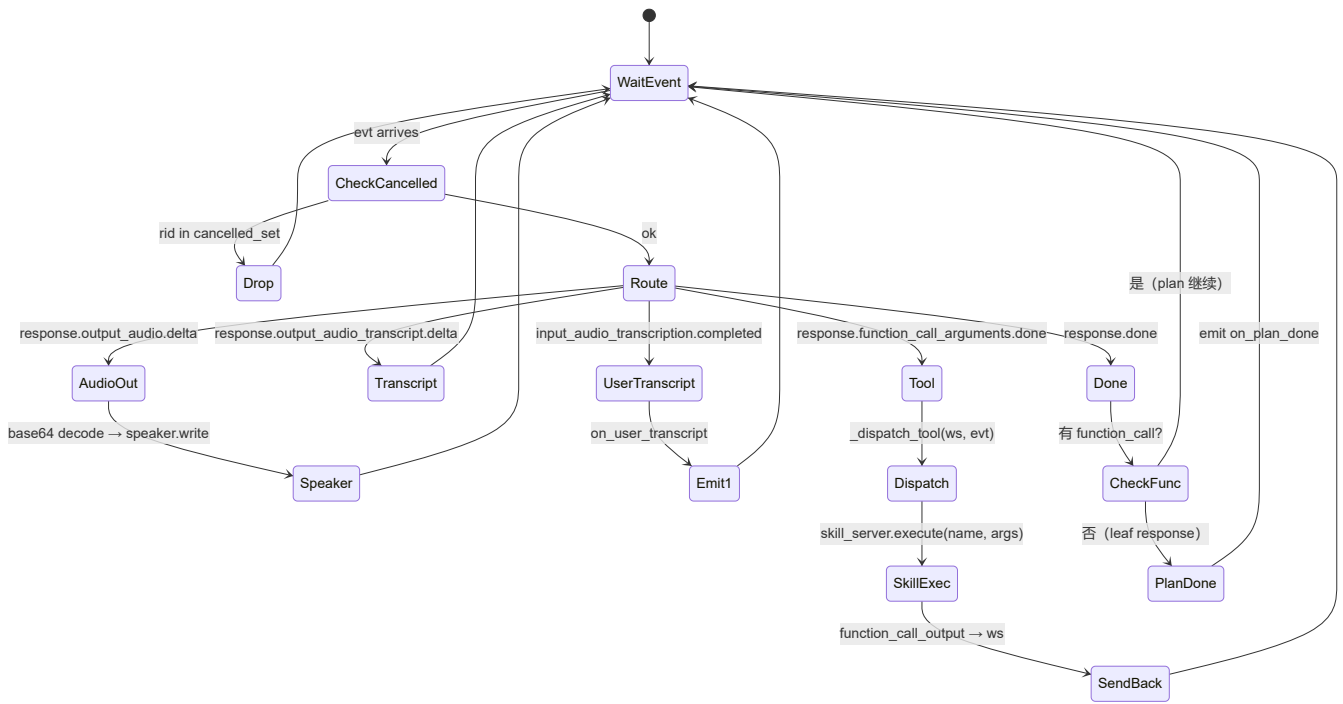
这正是用户的"perception 没有发挥作用"的直觉来源。Perception 实际上对 safety 和 watchdog 是连续生效的 (每次 motion validate 都会 snapshot)，但对快脑的"语义意识"是事件驱动+按需的。

6.3 快脑的 system prompt 结构

```
[REALTIME_SYSTEM_PROMPT_BRAIN]                                ← brain/prompts.py
+ "\n\n"
+ memory_summary.md (最近 N session 摘要)                    ← memory/__init__.py
+ AGENTS.md (recall 操作手册)                                ← memory/storage.py
```

注意：这个拼接发生在 `await brain_agent.run()` 之前一次 (`agent_main.py:1096-1110`)。Session 一旦开启，prompt 就被烧进 Realtime session，不会再变。所以"持续给快脑灌 perception"在协议层就不容易做 (必须每 N 秒发 `conversation.item.create`)。

6.4 事件循环 (`_handle_event`)



`cancelled_response_ids` 是 barge-in 的关键防护——保留最近 16 个被取消的 `response_id`，晚到的事件被静默丢弃，避免：

- 老的 `audio.delta` 在 `speaker.clear()` 之后又写进扬声器
- 老的 `function_call` 在用户已经说"停"之后还触发 `walk`

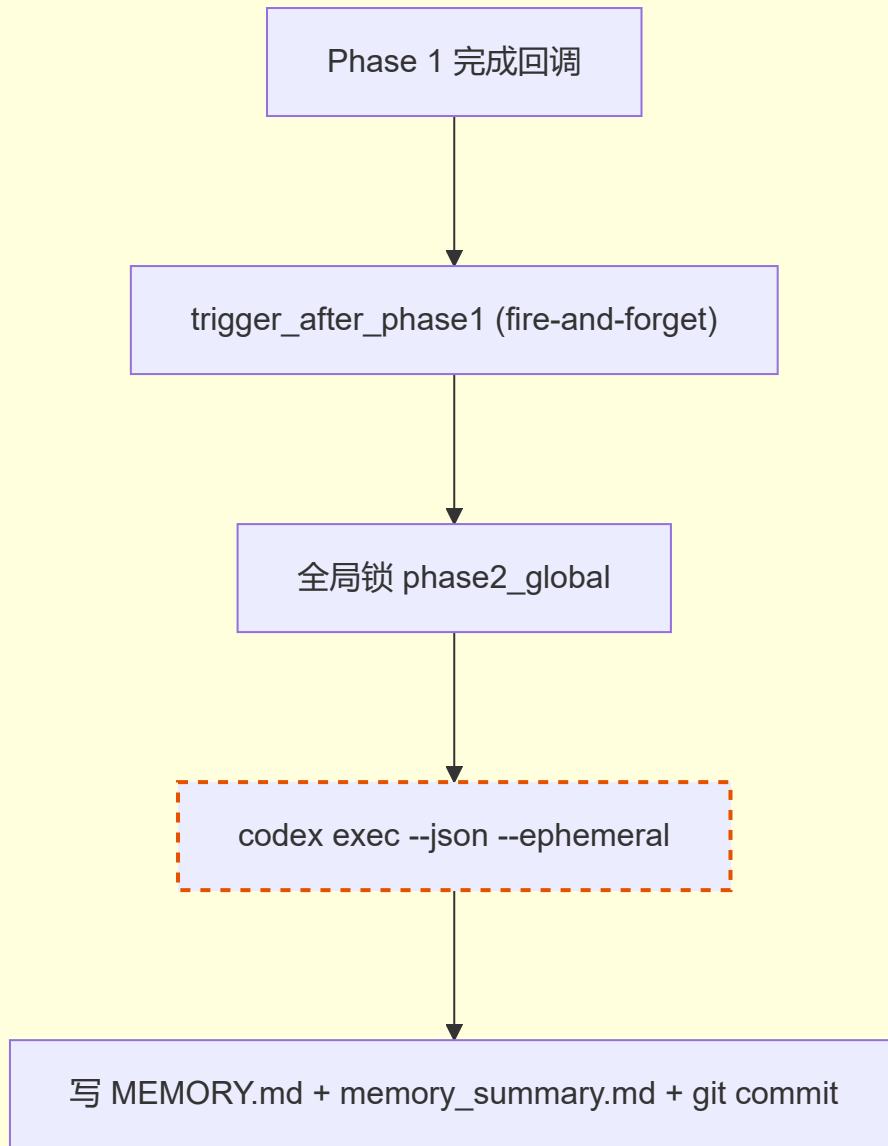
7. 慢脑 (Slow Brain) : Codex 的三种调用模式

用户的问题: `codex` 到底什么时候发挥作用?

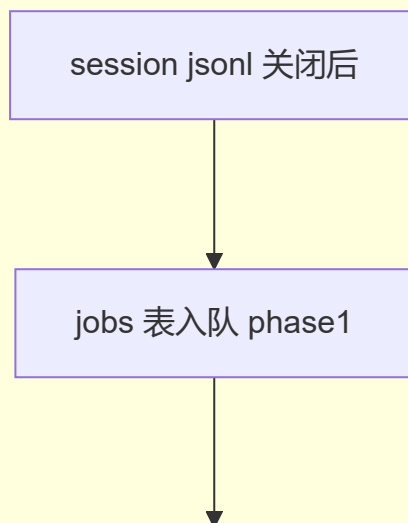
答案: 有三个完全独立的时刻。

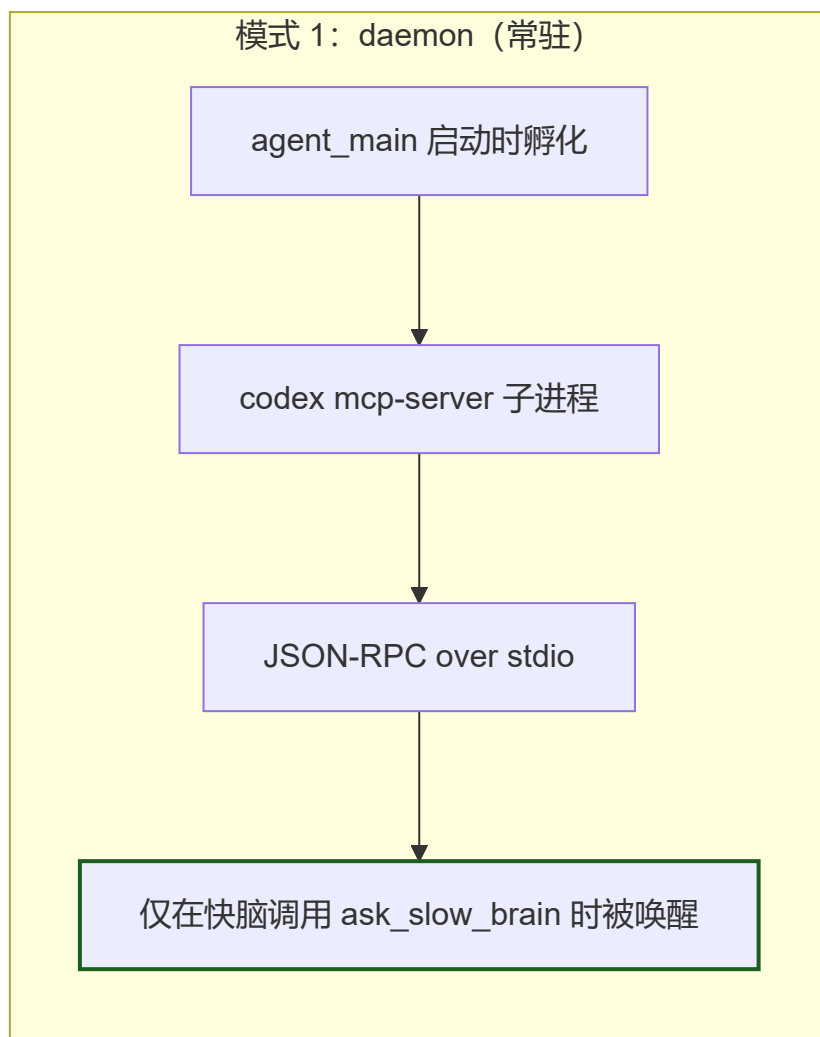
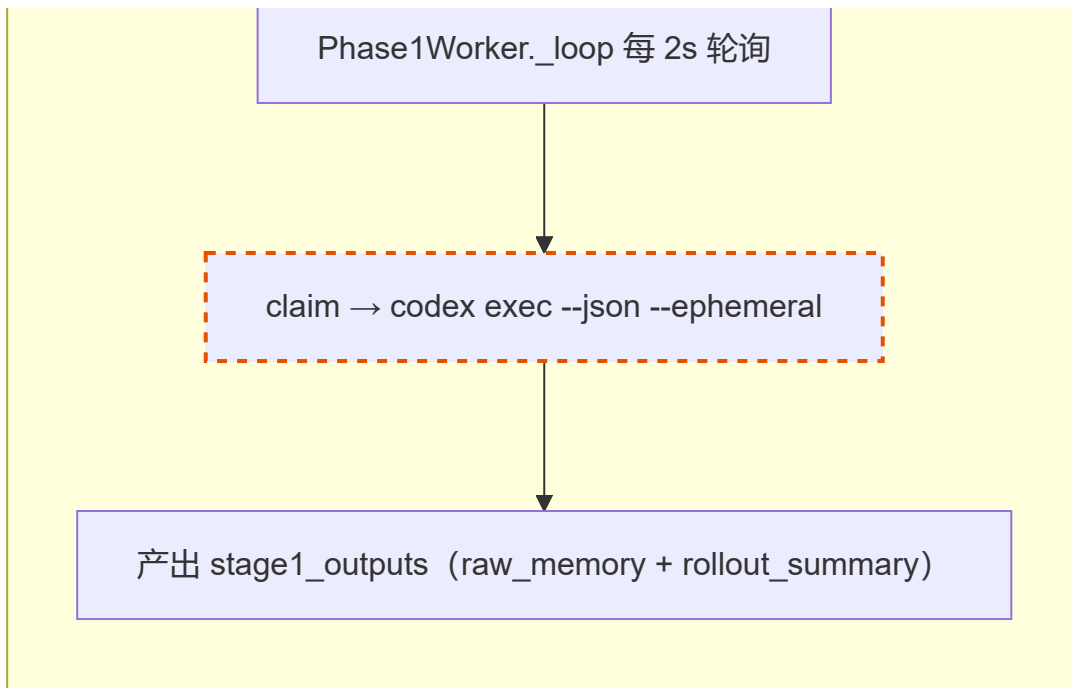
7.1 三种 codex 模式总览

模式 3: Phase 2 (全局合并)



模式 2: Phase 1 (每 session 一次)





7.2 模式 1: `codex mcp-server daemon`

孵化时机: `agent_main.py:1033` 调用 `await memory_subsystem.start()`, 内部 `asyncio.create_task(daemon.start())`。

命令行 (`daemon.py:417-435`) :

```
codex mcp-server \  
-c approval_policy=never \  
-c sandbox_mode="read-only" \  
-c model_reasoning_effort="high" \  
-c model_reasoning_summary="concise" \  
-c service_tier="fast"
```

关键事实:

- `effort=high` + `tier=fast` 是 **v1.1.0 的默认值** (`schemas.py:131-133`) , 来自用户记忆里"每次 codex 都要 high+fast"
- 守护进程**只通过** `ask_slow_brain(query)` **工具被快脑唤醒**——不会因为 session 结束、空闲超时、watchdog 等任何其他事件触发
- 子进程用 16 MB readline 缓冲 (`limit=self._stdout_buffer_bytes`) , 避免 codex 0.128 在长输出时把 readline 撑爆

协议: MCP JSON-RPC 2.0 over stdio。每次 `ask_slow_brain` 实际发的是:

```
{"jsonrpc": "2.0", "id": N, "method": "tools/call",  
  "params": {"name": "codex", "arguments": {"prompt": <wrapped>, "cwd": <memories_dir>}}}
```

`<wrapped>` 是 `_wrap_recall_prompt()` 加的 recall 前导 (`daemon.py:348-406`) , 告诉 codex:

- 用 `rg/cat/sed` 搜 `MEMORY.md` / `raw_memories.md` / `rollout_summaries/*.md` / 原始 `*.jsonl`
- ≤6 次 shell 调用就要给答案
- CJK 不要用 `\b`
- 答案 ≤120 中文字 / 80 英文词, 引用 `session_id` 8 字符前缀 + `turn_id`

7.3 模式 2 / 3: `codex exec --json --ephemeral`

这是与 daemon 完全不同的子进程 (`codex_client.py:105-126`) :

```
codex exec --json \  
--skip-git-repo-check \  
--ignore-user-config \  
-C <workdir> \  
-s read-only \  
-c approval_policy=never \  
-c model_reasoning_effort="high" \  
-c model_reasoning_summary="concise" \  
-c service_tier="fast" \  
--ephemeral
```

- 每次都都是一次性子进程 (spawn → 一段 prompt → stdin EOF → stdout 流 JSONL 事件 → 退出)
- `--ephemeral` 不写 `codex` 的对话历史
- Phase 1 和 Phase 2 走同一个 `CodexExecClient.exec()` 入口, 差异仅在 prompt + workdir

7.4 关键澄清: codex 何时不被调用?

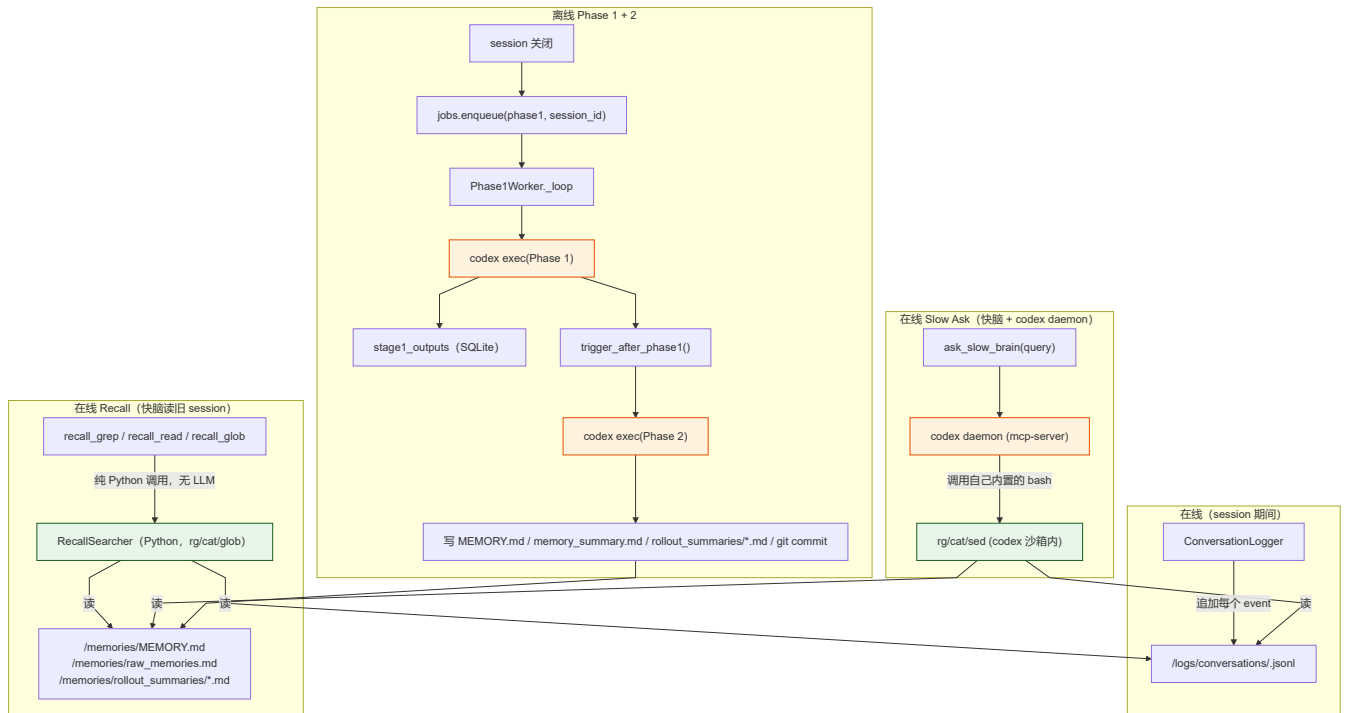
事件	是否触发 codex?
用户对快脑说一句话	✗
快脑调用 <code>walk / gesture / query_scene_state</code>	✗
快脑调用 <code>recall_grep / recall_read / recall_glob</code>	✗
快脑调用 <code>describe_scene</code>	✗ (用的是 OpenAI Vision, 不是 codex)
快脑调用 <code>ask_slow_brain</code>	✅ 模式 1 (daemon)
session 结束 (jsonl 关闭)	✅ 模式 2 (Phase 1) 入队 → 2 秒内取走
Phase 1 完成	✅ 模式 3 (Phase 2) fire-and-forget
Watchdog trip / FSM 转移 / E-stop	✗
周期性 cron	✗ — 没有定时任务, 完全事件驱动

用户的怀疑"我接入了 harness 但实际上 codex 没发挥作用"——部分成立:

- 在线对话期间, 只有显式 `ask_slow_brain` 才会动 `codex`
- 如果快脑从不主动调用 `ask_slow_brain` (比如 prompt 不提示它去用), 那么 daemon 会一直空闲
- 离线 Phase 1/2 是另一回事——只要有 session jsonl 关闭, 必然触发; 这是 harness 的"积累"路径

8. 记忆系统: Phase 1 / Phase 2 / Recall

8.1 三种数据流



8.2 Recall 的两条执行路径（用户的关键疑问）

用户原话："harness 机制的 memory recall 部分，我检查了一下日志好像也是 realtime 调用的 tool agent 执行的？"

部分对、部分错。让我们拆开：

路径 A：快脑直接调用 `recall_grep` / `recall_read` / `recall_glob`

```
# tool_schemas.py 把 recall_* 注册到 Realtime
# realtime_agent._execute_tool() -> skill_server.execute("recall_grep", args)
# skill_server._skill_recall_grep() -> memory.recall.grep(...)
# recall.py 直接调 `rg` 子进程 (或 grep / Python re)
# 没有 LLM。没有 codex。没有"tool agent"。
```

这条路径是**纯函数调用**：快脑 LLM 决定调用 → SkillServer 同步 dispatch → Python 调 `rg` → 返回 dict。**整条链路没有第二个 LLM。**

路径 B：快脑调用 `ask_slow_brain(query)`

```
# tool_schemas.py 注册 ask_slow_brain
# skill_server._skill_ask_slow_brain(query, timeout_s)
# memory.daemon.ask_slow_brain(query, ...)
# -> JSON-RPC tools/call -> codex mcp-server
# -> codex (这是一个独立的 LLM agent) 在自己的 sandbox 里跑 rg/cat/sed
# -> 返回 codex 综合后的答案 (≤120 中文字)
```

这条路径**才是"LLM-in-the-loop"**：快脑把球扔给 codex，codex 用自己的 reasoning + 自己的 shell tools 来检索。

所以**"realtime 调用的 tool agent"这个描述适用于路径 B，不适用于路径 A**。你看到的"realtime 工具执行 recall"如果具体是 `recall_grep` 这类工具，那就是 Python grep；如果是 `ask_slow_brain`，那才是 codex。

8.3 工程上"为什么这样设计"

工具	延迟	何时该用
<code>recall_grep</code> / <code>recall_read</code> / <code>recall_glob</code>	~10-50 ms	快脑已经知道关键词，要快速翻 MEMORY.md
<code>ask_slow_brain</code>	~3-15 s (含 codex reasoning)	快脑放弃了/找不到/需要综合多份 jsonl

`AGENTS.md` (recall 操作手册, 存在 `<robot>/memories/AGENTS.md`) 是给快脑读的"什么时候用哪个"——v1.1.0 默认快脑应该先 `grep`, 找不到才升级到 `ask_slow_brain`。

8.4 Phase 1 / Phase 2 输出契约

Phase 1 输出 (每 session 一份) :

```
{
  "raw_memory": "本 session 的关键事实条目 (中英文均可) ",
  "rollout_summary": "≤200 字的 session 摘要",
  "rollout_slug": "kebab-case-<10 字符>"
}
```

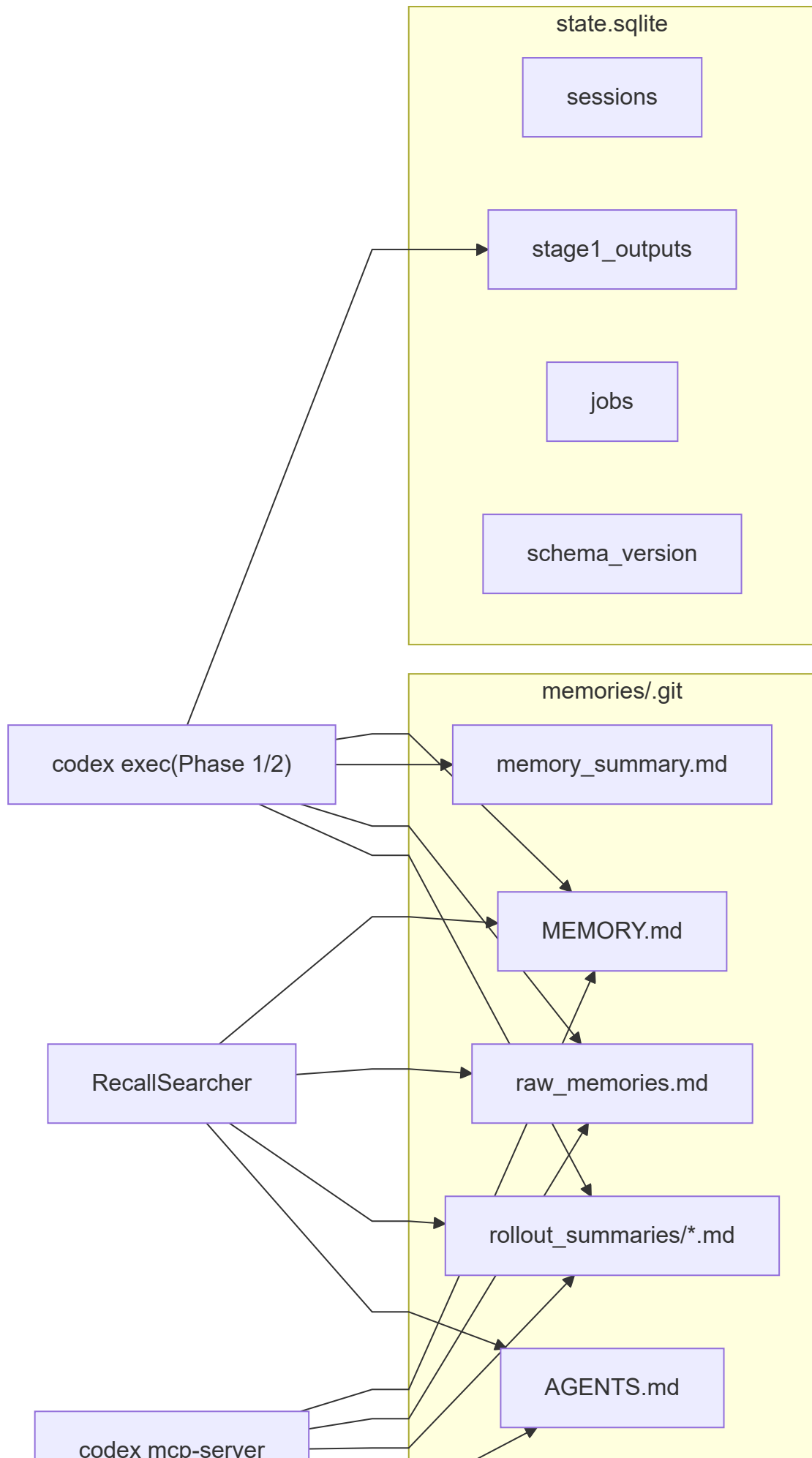
Phase 2 输出 (全局, 每次触发就重写) :

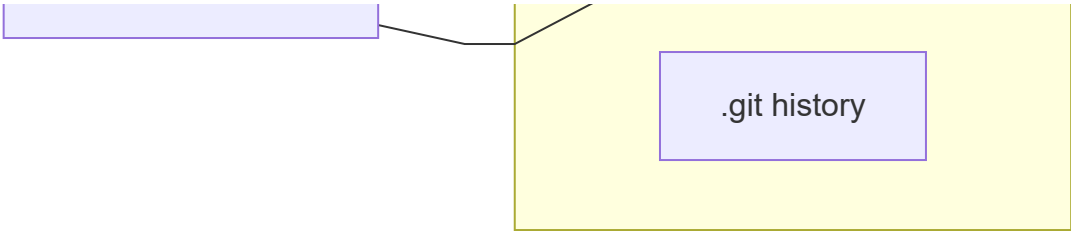
```
{
  "memory_md": "新的 MEMORY.md 全文 (curated) ",
  "memory_summary_md": "新的 memory_summary.md 全文 (concise digest) "
}
```

Phase 2 额外做:

- 把所有 stage1_outputs 的 raw_memory 拼成 `raw_memories.md` (确定性)
- 给每个 stage1_output 写一份 `rollout_summaries/<slug>.md`
- `git add . && git commit -m "phase2 @ <iso>"`, 保留每次合并的快照

8.5 SQLite + git 双轨持久化





9. 工具表（18 个 tool 的完整矩阵）

快脑实际能调用的所有工具（`tool_schemas.build_tool_schemas(sim=True, ...)`）。

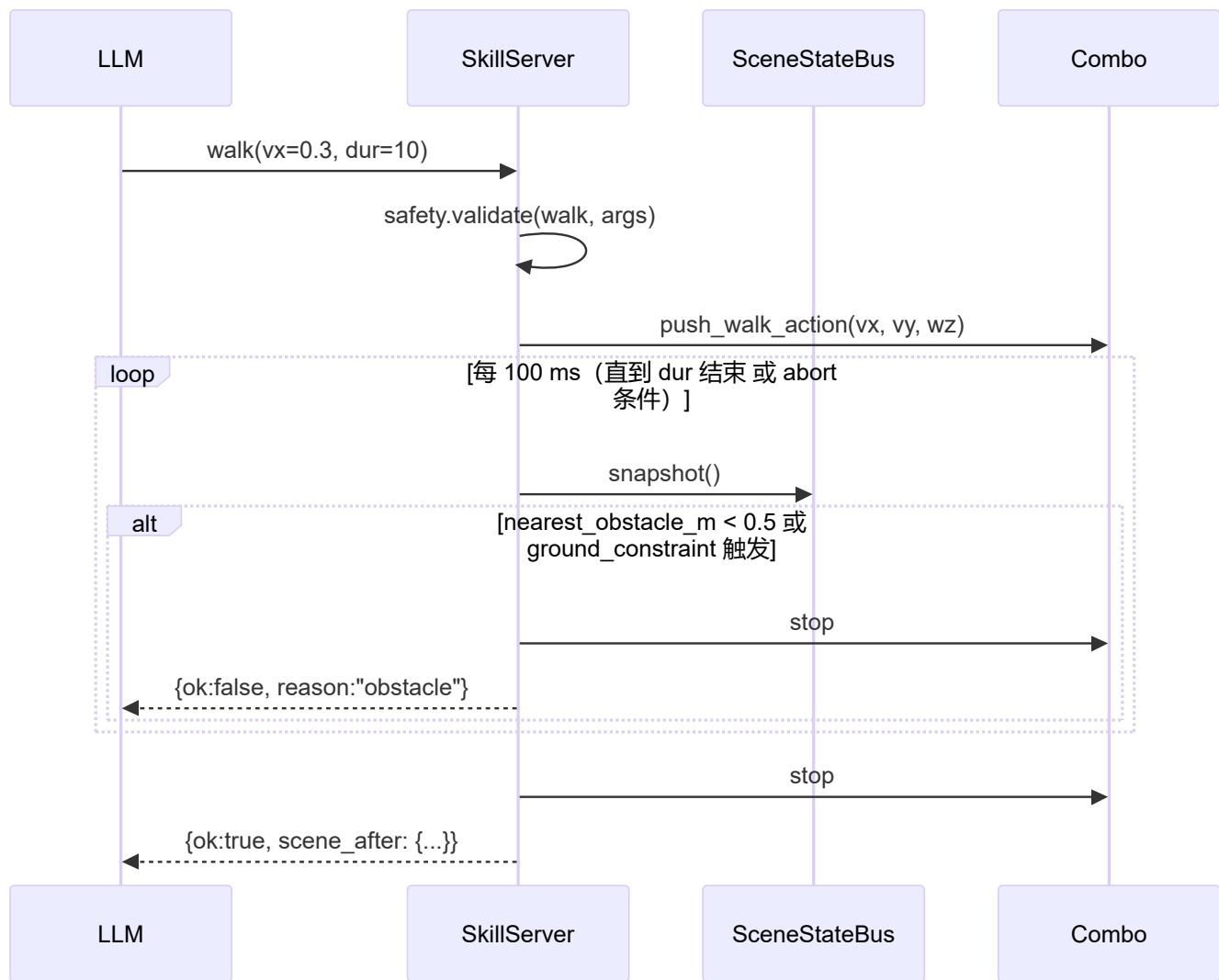
#	工具	层级	同步/异步	后端	是否进 safety.validate	在 vision_only 模式下保留？
1	<code>say(text)</code>	L1	同步	OpenAI TTS gpt-4o-mini-tts	否（直接 speaker）	是
2	<code>describe_scene(question, detail)</code>	L1	异步	OpenAI Vision gpt-5.5	否	是
3	<code>query_scene_state()</code>	L1	同步	SceneStateBus.snapshot()	否	是
4	<code>recall_history(kind, limit)</code>	L1	同步	直接读 jsonl	否	是
5	<code>look_at(target)</code>	L1 复合	异步	内部 → <code>turn</code>	是	是
6	<code>approach(target_distance_m)</code>	L1 复合	异步	内部 → 多次 <code>walk</code>	是	是
7	<code>mock_imitate(gesture)</code>	L1 复合	异步	内部 → <code>gesture</code> 列表	是	仅 <code>mock_imitation.enabled</code>
8	<code>ask_human(question)</code>	L1 复合	异步	<code>say</code> + 等待 5s	否	是
9	<code>recall_grep(pattern, scope, ...)</code>	L1 记忆	同步	RecallSearcher (rg / grep / re)	否	是
10	<code>recall_read(path, start, end)</code>	L1 记忆	同步	RecallSearcher (Path.read_text)	否	是
11	<code>recall_glob(pattern, limit)</code>	L1 记忆	同步	RecallSearcher (Path.glob)	否	是
12	<code>ask_slow_brain(query, timeout_s)</code>	L1 记忆	异步	codex mcp-server （高 effort，快 tier）	否	是
13	<code>walk(vx, vy, wz, duration_s)</code>	L2	异步	ComboController (reactive abort 100 ms 间隔)	是	否（vision_only 屏蔽）
14	<code>turn(yaw_deg)</code>	L2	异步	内部 → <code>walk(wz, dur)</code>	是	否
15	<code>gesture(name)</code>	L2	异步	Combo.push_arm_action (keyframe)	是	否
16	<code>static_pose(name)</code>	L2	异步	Combo.push_arm_action	是	否
17	<code>stop()</code>	L2	同步	combo halt + clear arm queue	是（但走快速路径）	是
18	<code>release_arms()</code>	L2	同步	combo arm release	是	否

真机独有（默认拒绝）：

- `loco_high(action)` — 仅当 `--real` 启用
- `arm_action_high(action_id)` — 仅当 `--real`
- `audio_tts_robot(text)` — 仅当 `--real`

9.1 motion tool 的"reactive abort"

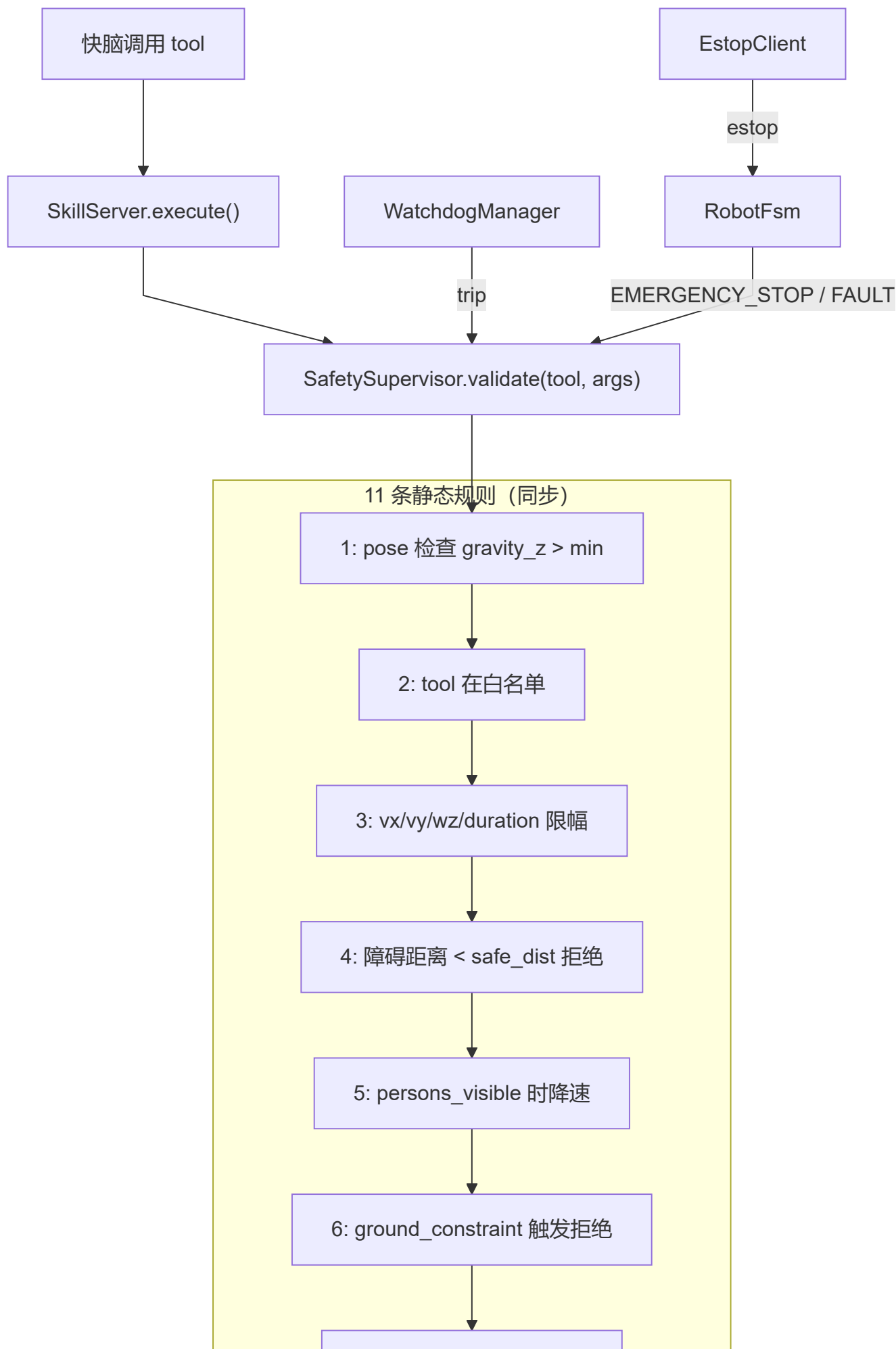
walk 不是单纯把 vx 写到 combo——skill_server._skill_walk() (line 583-633) :

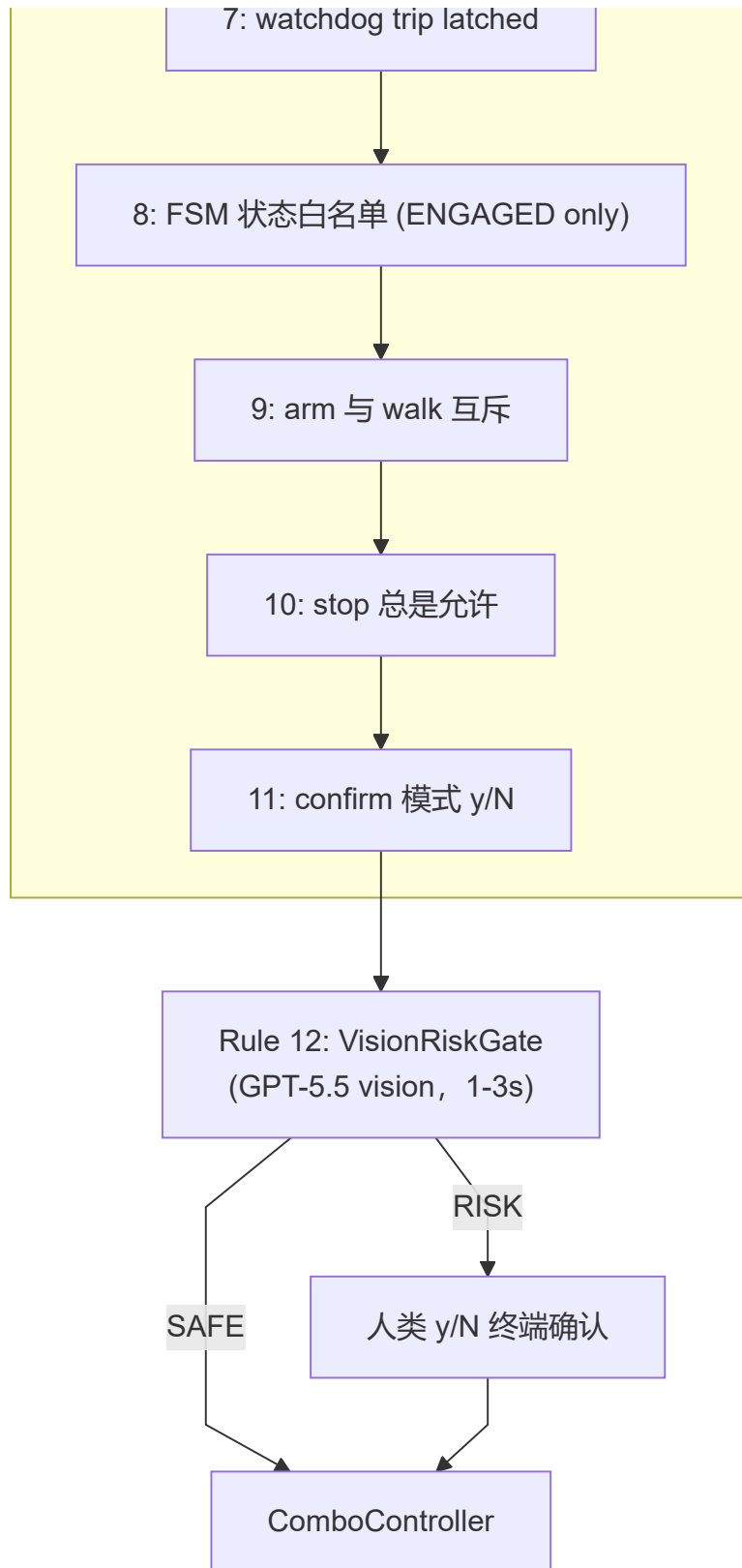


这是唯一一处在 tool 执行中持续读 perception 的代码，但消费者是 SkillServer，不是快脑 LLM。

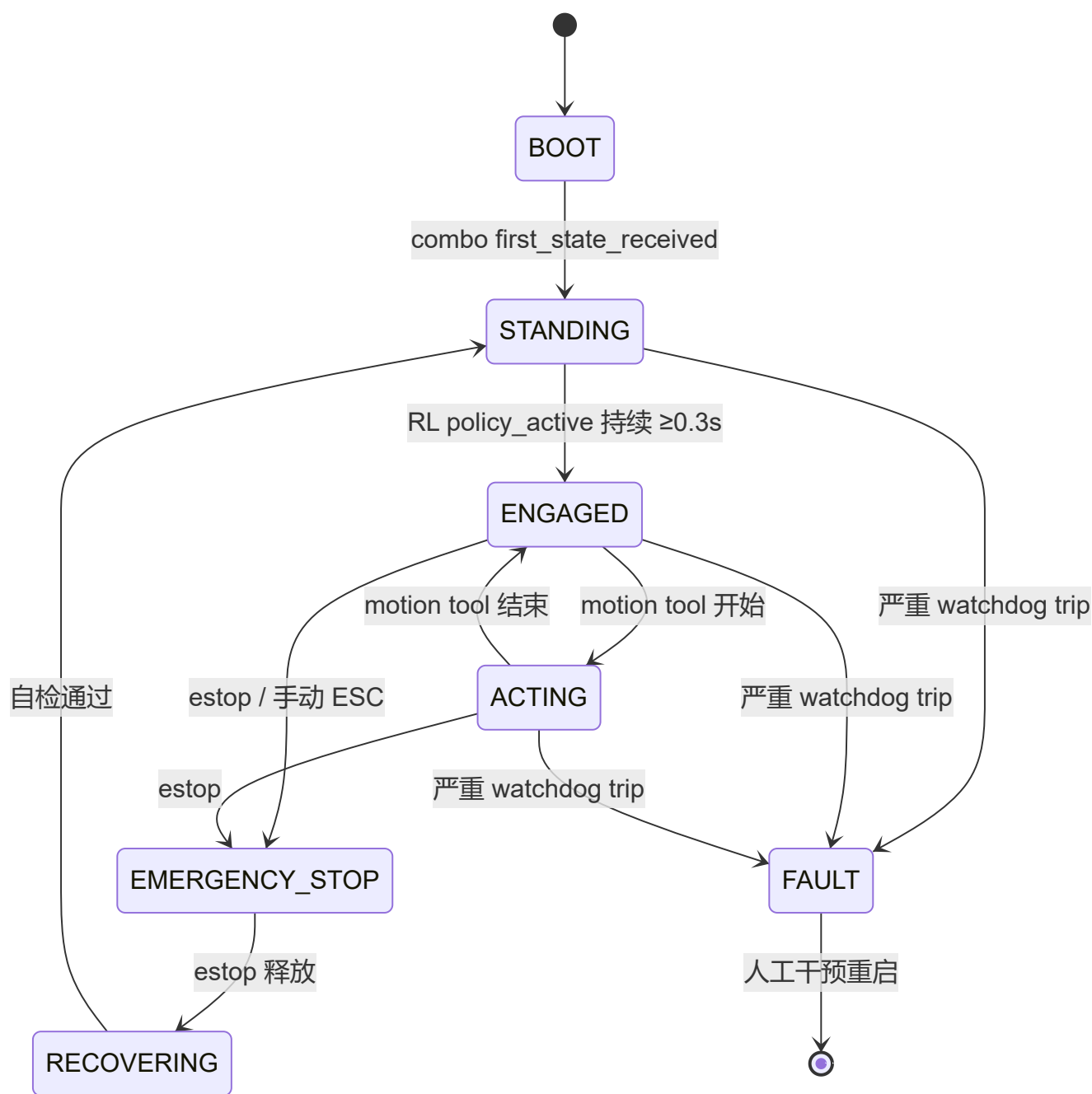
10. 安全监督：11+1 规则 + FSM + Watchdog

10.1 安全栈





10.2 FSM 7 态



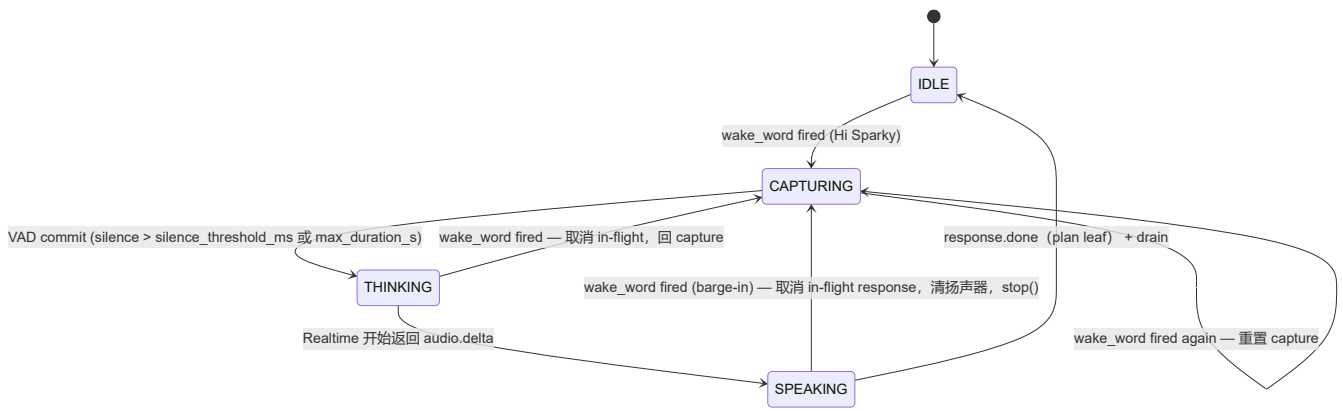
Safety 不允许任何 motion 在 BOOT / STANDING / EMERGENCY_STOP / FAULT 状态下执行。

10.3 E-stop 进程分离

EstopClient 在主进程轮询 /tmp/g1_brain_estop flag file (默认 safety.estop.flag_path)。

关键：E-stop 的"真正强制零扭矩"在另一个进程 estop_listener.py (独立 systemd / 终端启动)，它直接 publish rt/lowcmd 30 帧零扭矩，**不依赖主进程**。主进程死掉它也照样工作。

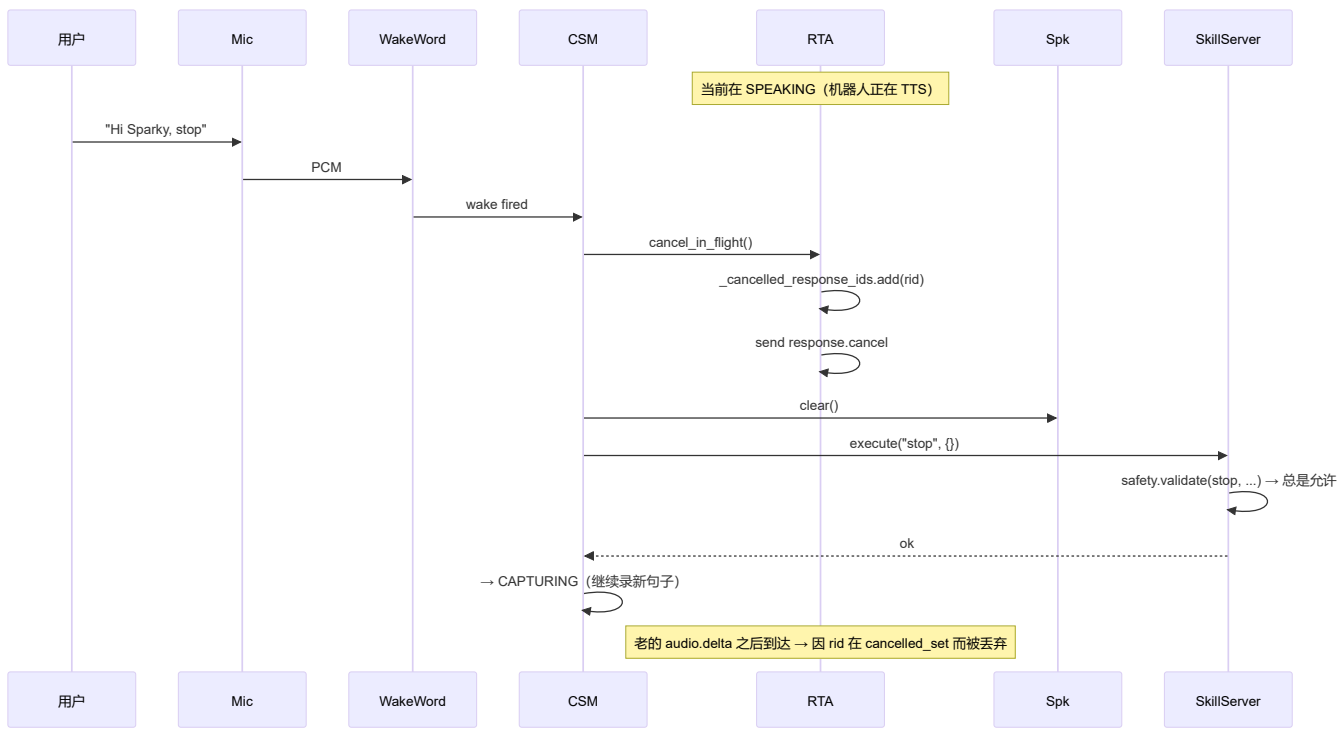
11. 音频流与对话状态机



11.1 唤醒词后端

- v1.1.0 默认 `wakeword.backend=openai` (OpenAI Transcribe `gpt-4o-transcribe`, 1.5s 滚动窗)
- 1300 Hz 入口 RMS 门控 `rms_threshold=300` (避免 TTS 自激发)
- AEC: 扬声器最近 `recent_played_rms(window_s)` 减去预测回声

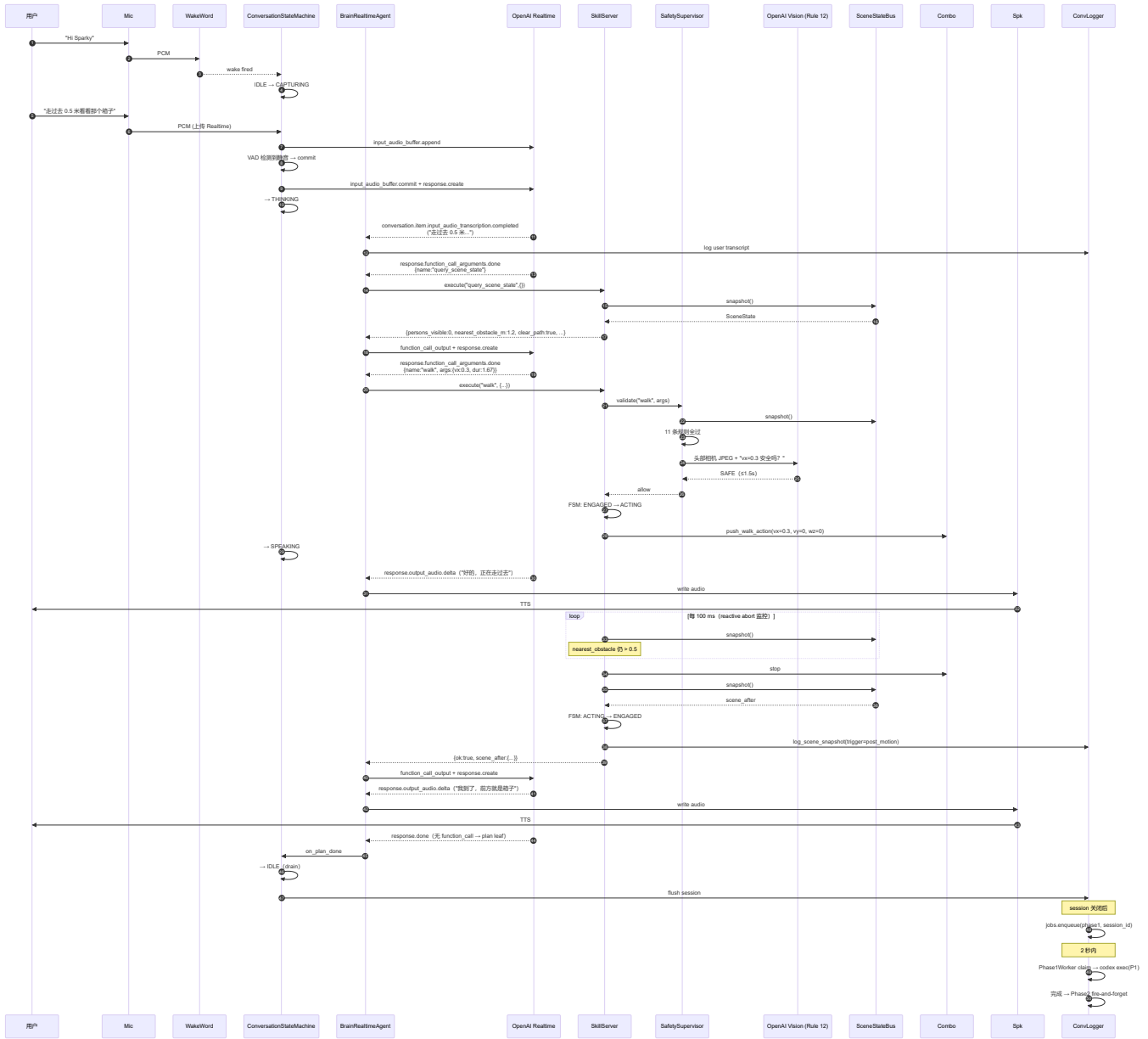
11.2 barge-in 路径 ("Hi Sparky" 在任意状态下都能打断)



用户的最新 patch (3b7fff6、74722b3、a5383df、cf09042、42597de 这几条 commit) 就是修这条路径的 RMS 门控和 AEC 延迟。

12. 一次完整 turn 的端到端时序

用户说"Hi Sparky, 走过去 0.5 米看看那个箱子" 的端到端拆解。



这一个 turn 触及了：

- 4 个 LLM 服务 (Realtime / Vision / TTS / Codex Phase 1/2)
- 18 个工具中的 3 个 (query_scene_state / walk / 隐含 say via TTS delta)
- 2 个进程 (main + combo)
- 后台一次 Phase 1/2 异步消化

13. 关键问题回答

Q1：快脑的本地 perception 是不是没有发挥作用？

结论：部分成立，对"快脑的语义意识"是 partial。

维度	Perception 是否生效
Safety（每次 validate 都 snapshot）	✅ 持续生效

维度	Perception 是否生效
Watchdog (10 Hz 读 frame_age)	✅ 持续生效
Reactive abort (walk 期间每 100 ms 检查)	✅ 持续生效
Motion tool 的 scene_after 注入	✅ 每次 motion 之后
快脑主动决策 ("我是不是该看一眼")	⚠️ 依赖 LLM 自己决定调用 query_scene_state
持续视觉流 (YOLO 帧逐帧进 LLM)	❌ 没有这条路径

也就是说：

- 你的 YOLO/Pose/depth/ground 全都在 5-15 Hz 运行
- 但快脑 LLM 收不到它们的实时输出
- 它只在两个时刻"看一眼世界":
 1. 自己主动调用 query_scene_state / describe_scene
 2. 上一个 motion tool 返回的 scene_after 字段

如果你的 system prompt 没有告诉 LLM "动作前先 query_scene_state", 它就真的不会去看。这是用户感觉"perception 没在用"的根因。

如果你想让 perception 持续推送给快脑，需要做的事情之一：

- 在主循环开一个 ticker，每 N 秒（比如 5s）调用
RTA.inject_perception_event(scene.snapshot().summary_for_llm())
- 但这会显著拉高 Realtime token 成本，需要权衡

Q2: harness 的 memory recall 是不是 realtime tool agent 实时调用？

部分对、部分错——取决于你看的是哪个工具。

- 如果日志里看到的是 recall_grep / recall_read / recall_glob —— 纯 Python，没 LLM，是 RecallSearcher 直接 rg / cat / glob
- 如果看到的是 ask_slow_brain —— 是 codex mcp-server，那是一个真正的 LLM agent，它在自己的 sandbox 里跑 rg + cat + sed

所以"realtime 调用的 tool agent"这句话本身不准确：v1.1.0 里只有 ask_slow_brain 是 agent；其他 recall_* 是裸函数。

Q3: Codex 到底什么时候发挥作用？

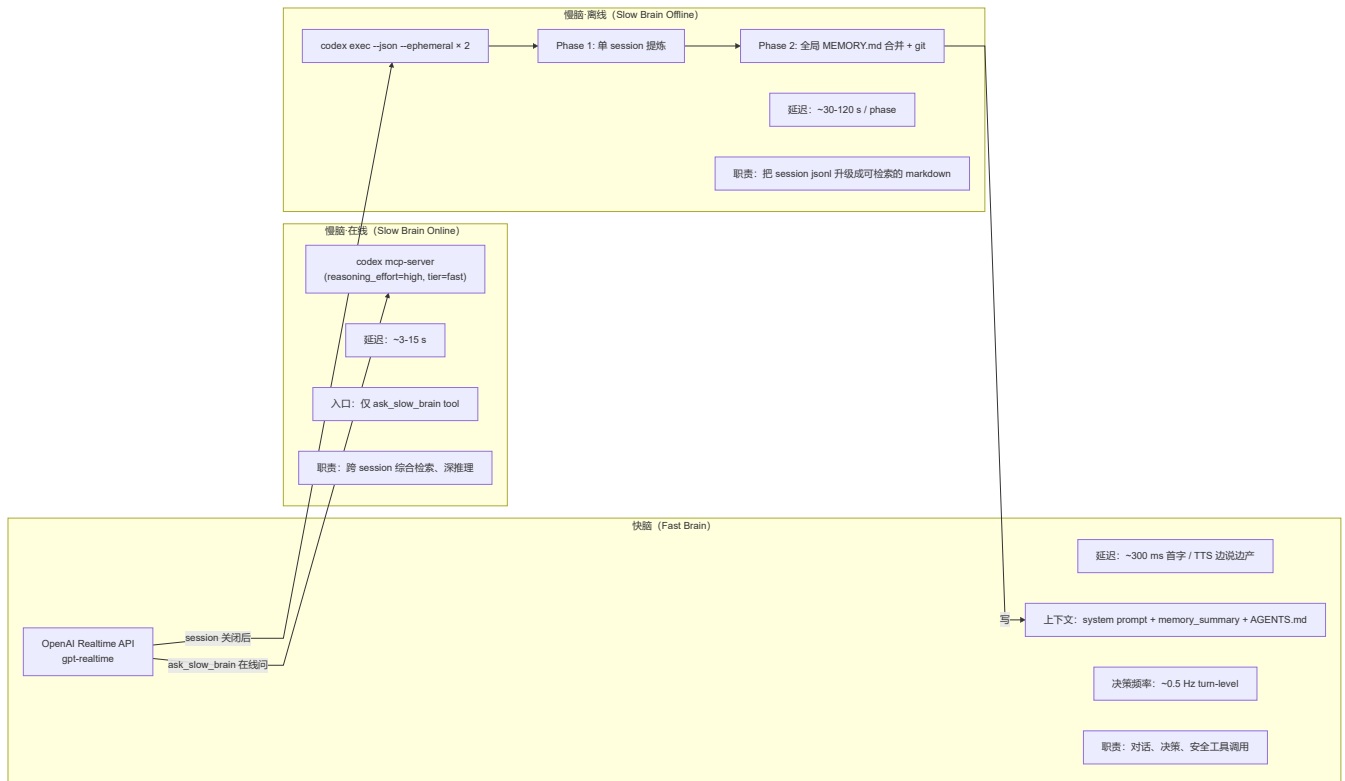
三个时刻，且只有这三个时刻：

1. 快脑主动调用 ask_slow_brain (codex mcp-server daemon) —— 在线、按需、最常见的可视化路径
2. session 关闭后入队 Phase 1 (codex exec --json --ephemeral) —— 离线、必然、每 session 一次
3. Phase 1 完成回调 trigger Phase 2 (codex exec --json --ephemeral) —— 离线、必然、每次 P1 完成都触发

Codex 不参与的事（用户可能误以为参与）：

- 不参与每个对话 turn 的决策（那是 Realtime LLM）
- 不参与 `recall_grep` 等纯函数工具
- 不参与 `describe_scene`（那是 OpenAI Vision，与 codex 完全独立）
- 不参与 safety validate（那是 Python 规则 + 可选 GPT-Vision）
- 不参与 perception（perception 全是本地小模型 + 数值算法）

Q4：快慢脑目前的设计是什么？



v1.1.0 的核心解耦哲学：

- 快脑做"响应快、决策粗"——端到端 < 1s，错了可以再问
- 慢脑做"思考慢、综合细"——容许 10s+，必须给出可引用的答案
- 两者共享 `memories/` 目录作为单一真相源——快脑通过 `grep` 直接读，慢脑通过 `codex` 综合产出

14. 已知局限与下一步

14.1 v1.1.0 已经实证可用的部分

- ☒ Realtime 快脑 + 18 工具
- ☒ SafetySupervisor 11+1 规则 + FSM + Watchdog
- ☒ Phase 1 + Phase 2 离线消化
- ☒ codex daemon `ask_slow_brain` 工具
- ☒ `recall_*` 纯函数工具
- ☒ Hi-Sparky 唤醒词 + barge-in（包括 SPEAKING 中断）

-  ComboProxy 子进程隔离

14.2 已知局限（与用户怀疑一致）

局限	影响	缓解
快脑无 perception 持续流	LLM 对世界的"持续感"全依赖它自己的 query 决策	1) Prompt 引导更勤地调用 <code>query_scene_state</code> 2) 加 ticker push <code>inject_perception_event</code>
perception 对快脑不可见的原始检测	YOLO 类别 / 置信度 / 框坐标都被压缩成 <code>summary_for_llm()</code> 5-6 个字段	加专门 tool <code>query_detections_raw()</code>
codex daemon 不会自发"主动思考"	必须快脑主动 ask	设计上是对的——避免空转 token，但要确保 prompt 指导快脑何时 ask
Phase 2 重写式合并	每次都全文重写 MEMORY.md，git diff 巨大	后续可以改成增量 patch
ConversationLogger 与 Phase 1 的时序	jsonl 必须完整 flush 之后才能消化，所以 shutdown 给 memory 留了 30s	已在 agent_main.py:1234 配置

14.3 如果你想让"快脑真正消费 perception"

最小入侵的改法（不展开实现，只列方向）：

1. 在 `agent_main.py:1140` 之后加一个 asyncio task：每 N 秒 `await brain_agent.inject_perception_event(scene.snapshot().summary_for_llm())`
2. 在 system prompt 加一行 `"You will receive periodic perception summaries as system messages; treat them as context, don't always respond."`
3. 在 `BrainRealtimeAgent.inject_perception_event` 里加一个"去重"检查（只在场景变化 > 阈值时发送），避免 token 浪费

或者反过来——保留按需模式，但在 **system prompt 强化"动作前必须 query"** ——这样不增加 token 成本，只是让 LLM 自觉调用。

14.4 如果你想让"codex 更主动"

可考虑：

- 在 jsonl 写入达到阈值时自动 ask_slow_brain 做"中期反思"（事件驱动，不是 cron）
- 让 codex daemon 在 FAULT / EMERGENCY_STOP 这种异常 FSM 转移时被自动 ping（提供事后分析）

但**注意**：这些都会增加 token / 延迟，与"慢脑是按需脑"的设计哲学冲突。先确认必要性再做。

15. 电话桥 (Twilio + Realtime, 2026-05-24 增量)

本节是 v1.1.0 主版本之后的增量, 覆盖 `g1_brain/g1_brain/phone/` 子包与跨进程协作。
设计稿: 根目录 `mcp_twilio_design.md` (1765 行) ·
Spec: `docs/superpowers/specs/2026-05-24-twilio-realtime-phone-bridge-design.md` ·
实施计划: `docs/superpowers/plans/2026-05-24-twilio-phone-bridge.md` ·
实测通过日期: 2026-05-24 (commit `8c8fd5b`) 。

15.1 设计原则

原则	落地
安全侧零拷贝	电话端 tool call 直接进入既有 <code>SkillServer.execute()</code> —— 不复制 <code>SafetySupervisor</code> 、不复制 <code>vision_risk_gate</code> 、不复制 <code>SkillServer</code> 。
进程内桥接	<code>bridge_server</code> 与 <code>brain</code> 同进程同 <code>asyncio</code> 循环; tool 调度是 in-process method call。
公网零暴露	<code>bridge</code> 只监听 <code>127.0.0.1:8787</code> , 永远不绑 <code>0.0.0.0</code> ; 公网入口走 VPS nginx + autossh 反向隧道。
双麦克风互斥	<code>/tmp/g1_brain_voice_lease</code> (fcntl flock'd JSON) 确保本地话筒和电话不会同时驱动机器人。
Fail-closed	<code>safety.vision_gate.enabled=false</code> → 桥拒绝启动; Twilio 签名错 → 403; caller 不在白名单 → 立即挂; <code>start_phone_call</code> 目标不在白名单 → 拒拨。

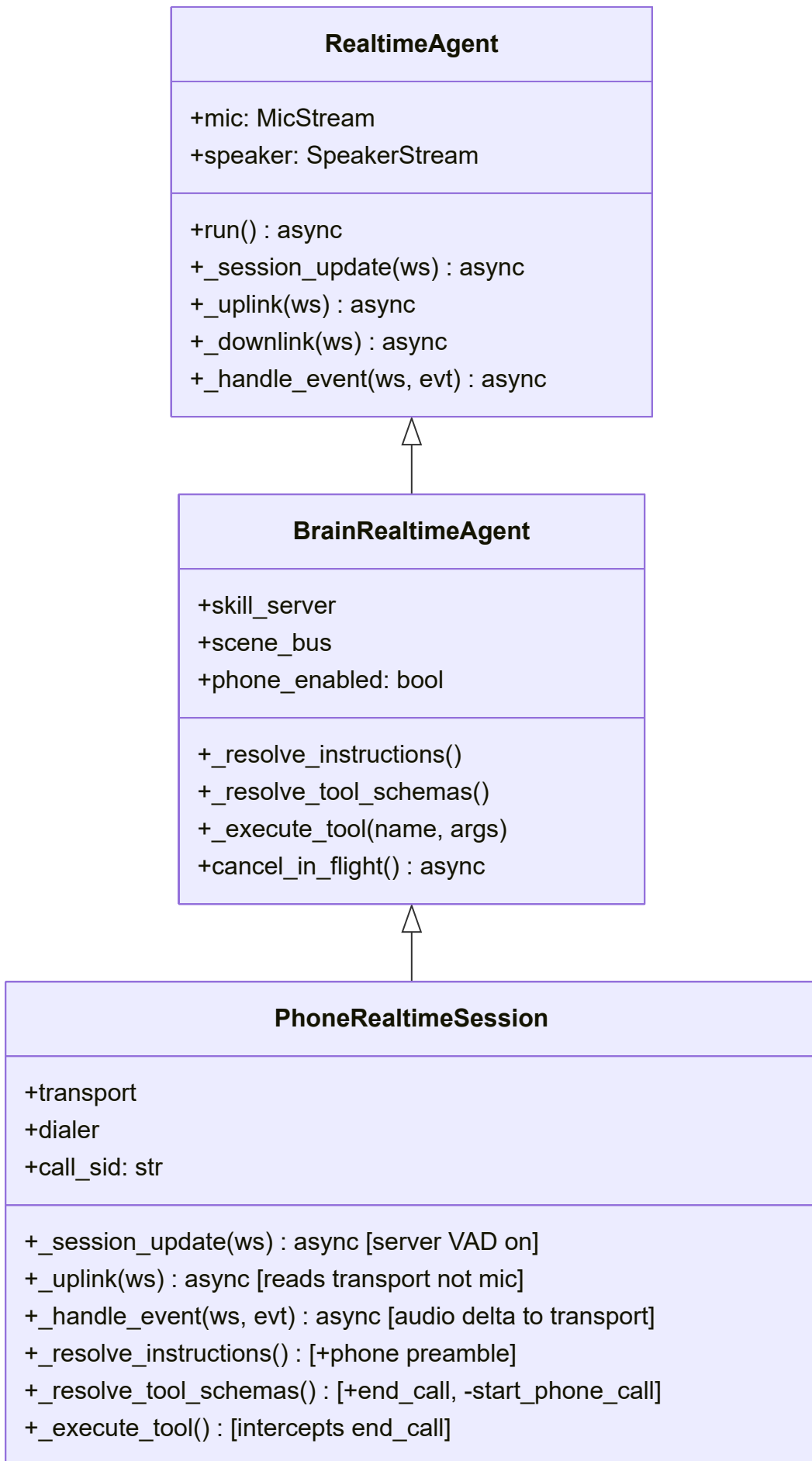
15.2 任务树扩展

`agent_main --enable-phone` 在原 v1.1.0 任务树之上追加一个 `asyncio task` 和一组按需子任务:

```
agent_main 主进程 (原 v1.1.0)
|
├─ BrainRealtimeAgent (Fast Brain, 本地话筒) ..... §6
├─ CodexDaemon (Slow Brain online) ..... §7
├─ Phase1Worker / Phase2Worker ..... §8
├─ ComboProxy 子进程 ..... §3
├─ CameraHub / YOLO / Pose / watchdog ..... §3
|
└─ NEW phone/bridge_server.py
    └─ aiohttp.web.AppRunner @ 127.0.0.1:8787
        ├── GET /healthz — 始终在
        └─ GET /twilio (Upgrade)
            └─ 每次来电 spawn 一组任务 (call lifetime) :
                ├── PhoneRealtimeSession.run()
                │   ├── _uplink(ws) — Twilio media → OpenAI
                │   └─ _downlink(ws) — OpenAI events → _handle_event
                └─ _kick (response.create greeting)
```

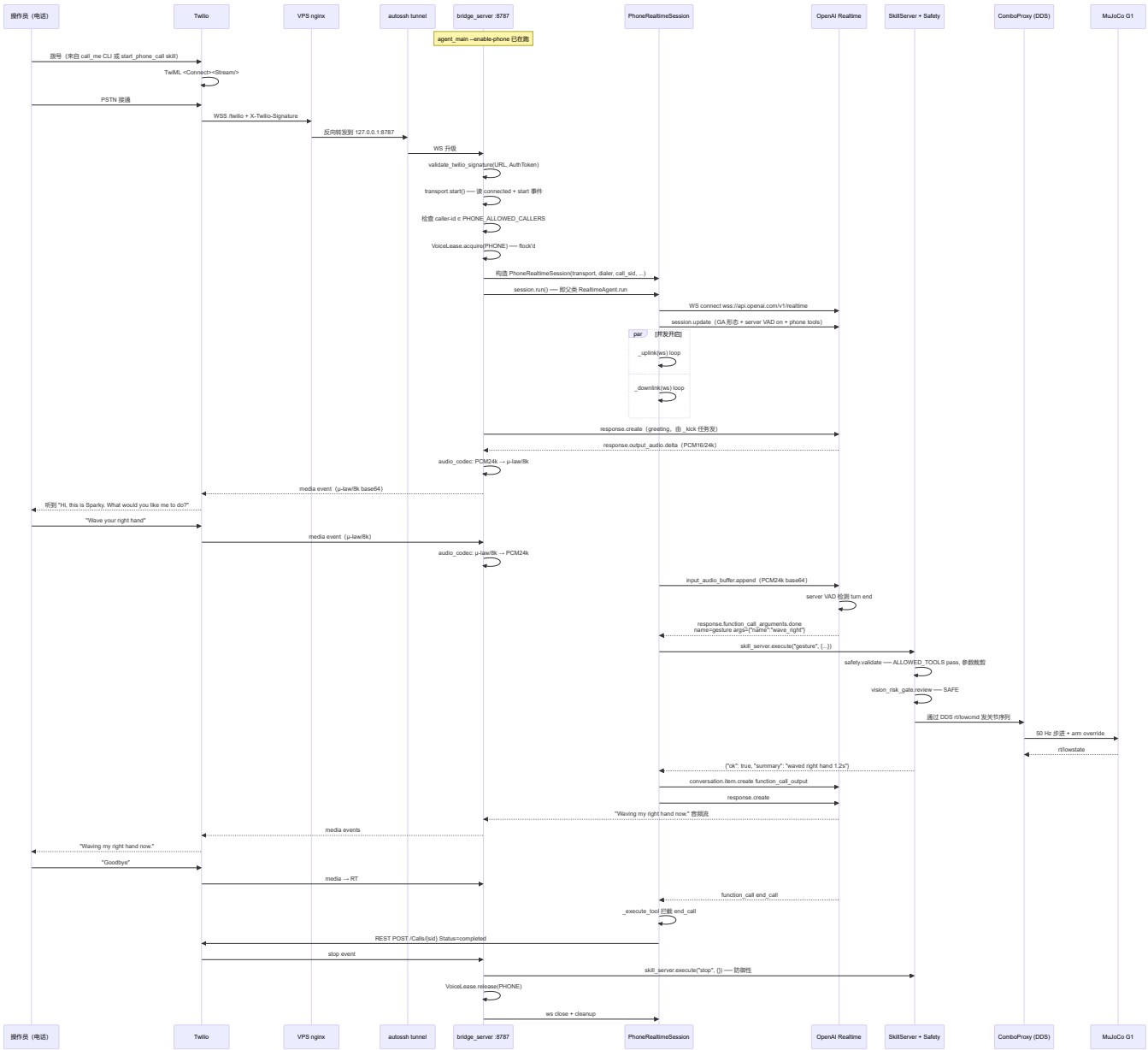
来电结束时 `finally` 子句保证: (1) 防御性 `skill_server.execute("stop", {})`; (2) `voiceLease.release(PHONE)`; (3) `transport.close()`。

15.3 PhoneRealtimeSession 继承图



覆盖原则：只覆盖 5 个方法 + 加 3 个新 dataclass 字段。 SkillServer.execute、 SafetySupervisor.validate、 vision_risk_gate.review、 conversation_logger、 plan tracker、 barge-in 取消逻辑——统统继承不动。

15.4 一次电话调用的端到端时序



15.5 子模块清单

文件	行数	职责
phone/config.py	~95	Pydantic TwilioConfig + PhoneConfig; load_from_env() 失败立即 raise PhoneConfigError。
phone/audio_codec.py	~95	mulaw8k_to_pcm24k / pcm24k_to_mulaw8k (scipy resample_poly 整数比 1:3 / 3:1, np.clip 防 int16 wrap) + StreamingResampler 持帧。

文件	行数	职责
phone/voice_lease.py	~145	VoiceLeaseManager{LOCAL_MIC, PHONE}; /tmp/g1_brain_voice_lease + fcntl.flock(LOCK_EX); stale 自动回收 (默认 1 h)。
phone/tunnel_health.py	~50	validate_twilio_signature() HMAC-SHA1 constant-time; build_healthz_payload()。
phone/twilio_dialer.py	~135	REST dial(to) / hangup(sid) / dry_run(); 默认 Account SID + Auth Token 鉴权 (API Key 路径见 _auth() 注释)。
phone/twilio_transport.py	~115	aiohttp WS 协议适配: start() 读 connected + start → startEvent; iter_inbound_pcm24k() 异步迭代; send_outbound_pcm24k() 走 StreamingResampler; clear_outbound() 给 barge-in 用。
phone/realtime_session.py	~155	PhoneRealtimeSession (继承 BrainRealtimeAgent); 覆盖 _uplink / _handle_event(audio delta + speech_started) / _session_update (server VAD on) / _resolve_* / _execute_tool(end_call); END_CALL_SCHEMA。
phone/bridge_server.py	~240	build_app(...) → aiohttp; /healthz + /twilio WS 路由; _build_phone_session 注入 safety=skill_server.safety (不是 MagicMock, 2026-05-24 修过一次真火事故)。
phone/call_me.py	~55	CLI python -m g1_brain.phone.call_me [--to ...] [--dry-run]。

15.6 跨文件改动

文件	改动
brain/realtime_agent.py	添加 phone_enabled: bool = False 字段; _resolve_tool_schemas() 把它传给 build_tool_schemas(phone_enabled=...)。
brain/prompts.py	新增常量 PHONE_CALL_PREAMBLE (指定语言 = caller 用什么回什么, 默认英文)。
skills/tool_schemas.py	新增 START_PHONE_CALL_SCHEMA + END_CALL_SCHEMA; build_tool_schemas 加 phone_enabled kwarg; only-enable-when-true append start_phone_call。
skills/skill_server.py	__init__ 加 dialer=None, default_phone_to=None; _skill_start_phone_call(*, to=None); _allowed_phone_callers 白名单守门 (2026-05-24 真火 ASR 听错事故后加)。
safety/supervisor.py	ALLOWED_TOOLS_NO_MOTION 加 start_phone_call 与 end_call; _sanitize_no_motion 加对应分支。

文件	改动
apps/agent_main.py	新增 --enable-phone flag; 当开启时: load_phone_env → 强制要求 safety.vision_gate.enabled=true → TwilioDialer → 在已构造的 skill_server 上 late-wire _dialer / _default_phone_to / _allowed_phone_callers → build_app → aiohttp.web.TCPSite 绑 phone.bind_host:bind_port (默认 127.0.0.1:8787) 。
configs/g1_brain.yaml	末尾追加 phone: 块; enabled: false 默认。
.env.example	新增 7 个 Twilio + 桥变量样板。
pyproject.toml	dependencies 加 aiohttp>=3.9, pydantic>=2.0, scipy>=1.11, twilio>=9.0。

15.7 安全栈在电话场景下的差异

规则	电话场景的处理
run_mode	强制 active; 电话上没法敲终端按 y/N。
Rule 12 (vision_risk_gate)	替代 y/N 提示。 safety.vision_gate.enabled 必须 true, 否则桥 fail-closed 拒绝启动。
Rule 1 (whitelist)	ALLOWED_TOOLS_NO_MOTION 已加入 start_phone_call + end_call。
Caller-ID	桥侧白名单: PHONE_ALLOWED_CALLERS 环境变量。
Dial 目标白名单	skill 侧: SkillServer._allowed_phone_callers。即使 wake-word ASR 听错 (真事故: +6848 听成 +6888), 也无法外拨陌生人。
Twilio HMAC	桥侧每次 WS upgrade 校验 x-Twilio-signature (HMAC-SHA1 over URL, constant-time compare) 。
Voice lease	/tmp/g1_brain_voice_lease, fcntl.flock'd JSON。LOCAL_MIC 与 PHONE 互斥占有 brain, stale 1 h 后可被抢占。

15.8 已知局限

- **并发电话**: bridge 同一时刻只接受一个 phone session (VoiceLease 单 PHONE 槽) ; 多操作员需要先扩 VoiceLeaseManager 与 _twilio_ws 的 active-call 计数器。
- **Inbound 来电**: 当前只做出站拨号; 要支持来电需要加 /twiml/inbound 路由并在 Twilio 控制台把号码 Voice URL 指过去。仍然要走 caller-id 白名单。
- **STT 听数字稳健性**: 本地 wake-word 用的 gpt-4o-mini-transcribe 对 +61411706848 这种长串数字的精度有限; 白名单兜底但用户体验仍可能"我让你打 8 你却说要打 8888"。提示工程 / DTMF / 短代码可能更稳。
- **trial-account preroll**: Twilio Trial 账号每次出站都会插一段"by upgrading press any key..."的留言; 升级账号即可消除。
- **跨地理/国际拨号**: Twilio 账号默认只开启美国/加拿大出站; 其它区 (如澳大利亚) 需到 console → Voice Geo Permissions 显式启用。

15.9 实测验证记录 (2026-05-24)

步骤	结果
1. tunnel 公网可达	<code>curl https://twilio.openproduct.cn/healthz</code> 从外网返回 200 + 正确JSON
2. Twilio 凭据 dry-run	<code>call_me --dry-run</code> 输出 "My first Twilio account"
3. 全栈启动	T1 sim + T3 brain+phone bridge 同时跑，日志见 "phone bridge listening on 127.0.0.1:8787"
4. 出站拨号	<code>python -m g1_brain.phone.call_me</code> 触发响铃
5. 音频桥（双向）	接通听到英文 greeting, "Say hello in French" 收到法语回复
6. 机器人在电话指令下动作	"Wave your right hand" → tool: gesture(wave_right) → safety pass → vision_gate SAFE → DDS → MuJoCo G1 真挥手

15.10 后续路标

- 让模型在通话开始时主动询问操作员姓名 / 短指令偏好（减少 STT 听错率）。
- 增加 DTMF 通道：电话按键 0=立即 E-stop, 1=确认, 2=取消。
- 与 memory 子系统对接：phone session 的 jsonl 走相同的 Phase 1 / Phase 2 流水线（理论上已经走了，因为 ConversationLogger 是 brain 共享的；待端到端测一次）。

附录 A：关键 file:line 索引

关注点	文件:行
主入口	<code>g1_brain/apps/agent_main.py:519-1238</code>
DDS 必须先于 CameraHub	<code>agent_main.py:595-598</code>
ComboProxy 启动	<code>agent_main.py:680-704</code>
MemorySubsystem.start	<code>g1_brain/memory/__init__.py:95-109</code>
Codex daemon 孵化命令	<code>g1_brain/memory/daemon.py:417-435</code>
<code>_wrap_recall_prompt</code> 前导	<code>daemon.py:348-406</code>
Phase1Worker._loop	<code>g1_brain/memory/phase1.py:321-487</code>
Phase 2 全局锁	<code>g1_brain/memory/phase2.py:115-129</code>
RecallSearcher	<code>g1_brain/memory/recall.py:31-180</code>
SkillServer.execute	<code>g1_brain/skills/skill_server.py:194-298</code>

关注点	文件:行
<code>scene_after</code> 注入	<code>skill_server.py:277-281</code>
<code>BrainRealtimeAgent._execute_tool</code>	<code>g1_brain/brain/realtime_agent.py:163-182</code>
<code>inject_perception_event</code>	<code>realtime_agent.py:531-559</code>
<code>cancel_in_flight</code> + <code>rid drop</code>	<code>realtime_agent.py:115-117, 199-210</code>
<code>PerceptionRunner</code>	<code>g1_brain/perception/runner.py:44-126</code>
<code>ground_loop</code>	<code>runner.py:170-205</code>
<code>SceneStateBus</code> snapshot	<code>g1_brain/scene_state/fusion.py</code>
RobotFsm 7 态	<code>g1_brain/safety/state_machine.py:84-...</code>
<code>EstopClient</code>	<code>g1_brain/safety/estop_client.py</code>
<code>_skill_recall_grep</code>	<code>skill_server.py:753-774</code>
<code>_skill_ask_slow_brain</code>	<code>skill_server.py:815-844</code>
配置默认值 (<code>effort=high</code> , <code>tier=fast</code>)	<code>g1_brain/memory/schemas.py:131-133</code>
 电话桥 入口 + 启动检查	<code>g1_brain/apps/agent_main.py:1055-1090</code>
 桥 aiohttp app builder	<code>g1_brain/phone/bridge_server.py:30-55</code>
 桥 WS handler (签名 + 白名单 + lease)	<code>g1_brain/phone/bridge_server.py:65-145</code>
 <code>PhoneRealtimeSession</code> 类	<code>g1_brain/phone/realtime_session.py:30-155</code>
 <code>_session_update</code> 覆盖 (<code>server VAD on</code>)	<code>g1_brain/phone/realtime_session.py:60-100</code>
 <code>audio_codec.StreamingResampler</code>	<code>g1_brain/phone/audio_codec.py:55-95</code>
 <code>VoiceLeaseManager.acquire</code>	<code>g1_brain/phone/voice_lease.py:67-95</code>
 <code>validate_twilio_signature</code>	<code>g1_brain/phone/tunnel_health.py:18-40</code>
 <code>TwilioDialer._auth</code> / <code>dial</code> / <code>dry_run</code>	<code>g1_brain/phone/twilio_dialer.py:30-130</code>
 <code>_skill_start_phone_call</code> + 白名单守门	<code>g1_brain/skills/skill_server.py:709-735</code>
 <code>ALLOWED_TOOLS_NO_MOTION</code> 含 <code>start_phone_call</code> + <code>end_call</code>	<code>g1_brain/safety/supervisor.py</code>
 <code>PHONE_CALL_PREAMBLE</code>	<code>g1_brain/brain/prompts.py:175-200</code>

附录 B: v1.1.0 与 v1.0 的 diff 速览

- 新增 12 号 Rule (GPT-Vision risk gate) —— 在 11 条静态规则之后、用户 y/N 之前

- 新增 `recall_grep` / `recall_read` / `recall_glob` / `ask_slow_brain` 工具
- 新增 `codex mcp-server` 常驻 daemon
- 新增 Phase 1 / Phase 2 离线消化管线 + `memories/.git`
- 新增 `ConversationLogger` (Claude-harness-compatible jsonl)
- 改造唤醒词: 从 `va-demo` 的 "only-in-IDLE" 改为 "any-state barge-in"
- 改造 audio control: `cleaned-RMS gate` + `AEC delay` (commit `74722b3`)

v1.1.1 增量 (2026-05-24, 电话桥)

- 新增 `g1_brain/phone/` 子包 (`config` / `audio_codec` / `voice_lease` / `tunnel_health` / `twilio_dialer` / `twilio_transport` / `realtime_session` / `bridge_server` / `call_me`) , 合计 ~1200 LOC + 46 测试。
- 新增 2 个 LLM 工具: `start_phone_call` (本地话筒侧, 带号码白名单守门) + `end_call` (电话 session 侧)。
- 新增 1 个 brain dataclass 字段: `BrainRealtimeAgent.phone_enabled` (通过 `agent_main` 的 `--enable-phone` 注入)。
- 改 `safety/supervisor.py::ALLOWED_TOOLS_NO_MOTION`: 加入 `start_phone_call` + `end_call`。
- 改 `brain/prompts.py`: 新增 `PHONE_CALL_PREAMBLE` (pin caller's language, 默认英文)。
- 改 `apps/agent_main.py`: `--enable-phone` flag + late-wire dialer + fail-closed on `safety.vision_gate.enabled=false`。
- 改 `configs/g1_brain.yaml`: 末尾追加 `phone`: 节。
- 改 `pyproject.toml`: 加 `aiohttp`, `pydantic`, `scipy`, `twilio` 依赖。
- 新增 systemd-user unit `sparkytun-tunnel.service` (autossh 反向隧道到 VPS)。
- 公网入口 `wss://twilio.openproduct.cn/twilio` (nginx + Let's Encrypt)。
- 实测通过: CLI 拨号 → 电话响铃 → 通话双向音频 → 电话语音指令 → MuJoCo G1 真挥手 (CallSid `Cab849c8e4eae9efcd5051f1e08f3e88e8` 等多次)。

本文件版本: 1.1.0-runtime (+ 1.1.1-phone-bridge 2026-05-24 增量)

生成时间: 2026-05-24

维护者: 作者本人; 如发现与代码不符, 以代码为准并提 issue。